

Unitree Week 8

Presentation by Mario Ortiz

This week

- Completed installing and bridging Isaac Sim + Isaac Lab
- Ran training program for Unitree G1 to walk in rough terrains, and was able to run simulation of final iteration of training, showing the G1 robot walking.

Log Notes

- Downloaded Isaac sim zip file, extracted zip file to “C:\User\Mario\isaac-sim” folder
- Cloned isaac lab
 - PS C:\Users\Mario> git clone <https://github.com/isaac-sim/IsaacLab.git>
- Bridged Isaac Sim with Isaac Lab
 - PS C:\Users\Mario> cd .\IsaacLab\
 - PS C:\Users\Mario\IsaacLab> cmd /c mklink /D _isaac_sim C:\Users\Mario\isaac-sim
- With the bridge, we do this to install python and other libraries from Isaac Sim onto Isaac Lab
 - .\isaaclab.bat --install
- Installing Reinforcement Learning directories

Log Notes

- Upgrading pip on isaac sim kit folder

& "C:\Users\Mario\isaac-sim\kit\python\python.exe" -m pip install --upgrade pip

- Reinstalling Isaac-sim

& "C:\Users\Mario\isaac-sim\kit\python\python.exe" -m pip install "isaacsim[all]==5.1.0" --extra-index-url <https://pypi.nvidia.com>

- Installing specifically torch 2.10.0

& "C:\Users\Mario\isaac-sim\kit\python\python.exe" -m pip install torch==2.10.0 torchvision==0.25.0 --index-url <https://download.pytorch.org/whl/cu126>

- Installing reinforcement learning library to isaac lab

cd C:\Users\Mario\IsaacLab
.\isaacclab.bat --install rsl_rl

Log Notes

```
rs_rl_ppo_cfg.py 2 X
C:\Users\Mario> isaacsim > source > isaacsim_tasks > isaacsim_tasks > manager_based > locomotion > velocity > config > g1 > agents > rs_rl_ppo_cfg.py > G1FlatPPORunnerCfg > _post_init_

1  # Copyright (c) 2022-2026, The Isaac Lab Project Developers.
2  # SPDX-License-Identifier: BSD-3-Clause
3
4  from isaacsim.utils import configclass
5  from isaacsim.rl.rl import RslRlOnPolicyRunnerCfg, RslRlPpoActorCriticCfg, RslRlPpoAlgorithmCfg
6
7  @configclass
8  class G1RoughPPORunnerCfg(RslRlOnPolicyRunnerCfg):
9      num_steps_per_env = 24
10     max_iterations = 3000
11     save_interval = 50
12     experiment_name = "g1_rough"
13
14     # This is the "Dual-Link" config to satisfy both Isaac Lab and RSL_RL
15     policy = RslRlPpoActorCriticCfg(
16         class_name="ActorCritic",
17         init_noise_std=1.0,
18         actor_obs_normalization=False,
19         critic_obs_normalization=False,
20         actor_hidden_dims=[512, 256, 128],
21         critic_hidden_dims=[512, 256, 128],
22         activation="elu",
23     )
24
25     algorithm = RslRlPpoAlgorithmCfg(
26         value_loss_coef=1.0,
27         use_clipped_value_loss=True,
28         clip_param=0.2,
29         entropy_coef=0.008,
30         num_learning_epochs=5,
31         num_mini_batches=4,
32         learning_rate=1.0e-3,
33         schedule="adaptive",
34         gamma=0.99,
35         lam=0.95,
36         desired_kl=0.01,
37         max_grad_norm=1.0,
38     )
39
40 @configclass
41 class G1FlatPPORunnerCfg(G1RoughPPORunnerCfg):
42     def __post_init__(self):
43         super().__post_init__()
44
45         self.max_iterations = 1500
46         self.experiment_name = "g1_flat"
47         self.policy.actor_hidden_dims = [256, 128, 128]
48         self.policy.critic_hidden_dims = [256, 128, 128]
```

- Needed to change the rs_rl_ppo_cfg.py program to match Isaac Lab's RL 4.0.1 requirements

Log Notes

- Setting python path to RL folder

```
$env:PYTHONPATH =
```

```
"C:\Users\Mario\IsaacLab\source\isaacrlab;C:\Users\Mario\IsaacLab\source\isaacrlab_tasks;C:\Users\Mario\IsaacLab\source\isaacrlab_rl"
```

- Command for train.py with rough environment: 1024 environments per iteration.

```
& "C:\Users\Mario\isaac-sim\kit\python\python.exe" scripts/reinforcement_learning\rsl_rl/train.py --task Isaac-Velocity-Rough-G1-v0 --headless --num_envs 1024
```

- Command for play.py with 32 g1 robots spawned in. (shows how the g1 robots move after final iteration from train.py command)

```
& "C:\Users\Mario\isaac-sim\kit\python\python.exe" scripts/reinforcement_learning\rsl_rl/play.py --task Isaac-Velocity-Rough-G1-v0 --num_envs 32
```

Training:

```
#####
Learning iteration 180/3000

Computation: 12188 steps/s (collection: 1.820s, learning 0.197s)
  Mean action noise std: 0.98
  Mean value_function loss: 0.0184
  Mean surrogate loss: -0.0198
  Mean entropy loss: 50.4388
  Mean reward: -4.98
  Mean episode length: 74.80
Episode_Reward/track_lin_vel_xy_exp: 0.0189
Episode_Reward/track_ang_vel_z_exp: 0.0124
  Episode_Reward/lin_vel_z_l2: 0.0000
  Episode_Reward/ang_vel_xy_l2: -0.0189
  Episode_Reward/dof_torques_l2: -0.0002
  Episode_Reward/dof_acc_l2: -0.0064
  Episode_Reward/action_rate_l2: -0.0281
  Episode_Reward/feet_air_time: 0.0002
Episode_Reward/flat_orientation_l2: -0.0043
  Episode_Reward/dof_pos_limits: -0.0010
Episode_Reward/termination_penalty: -0.2000
  Episode_Reward/feet_slide: -0.0050
Episode_Reward/joint_deviation_hip: -0.0038
Episode_Reward/joint_deviation_arms: -0.0067
Episode_Reward/joint_deviation_fingers: -0.0046
Episode_Reward/joint_deviation_torso: -0.0011
  Curriculum/Terrain_levels: 0.0034
Metrics/base_velocity/error_vel_xy: 0.1261
Metrics/base_velocity/error_vel_yaw: 0.5009
  Episode_Termination/time_out: 0.0000
  Episode_Termination/base_contact: 1.0000
-----
Total timesteps: 4448256
Iteration time: 2.02s
Time elapsed: 00:06:34
ETA: 01:42:26
```

Important Metrics:

- Mean reward: As this increases, it shows the robot is walking more.
- Mean episode length: Average time episodes takes; as it increases, it shows the robot is learning to walk more.
- Curriculum/terrain levels: Increases difficulty as iterations increase.
- Episode Termination/time_out: Shows percentage of robots being terminated per episode.

Training:

```
#####
Learning iteration 1009/3000

      Computation: 10790 steps/s (collection: 2.014s, learning 0.263s)
      Mean action noise std: 1.11
      Mean value function loss: 0.0272
      Mean surrogate loss: -0.0122
      Mean entropy loss: 49.2307
      Mean reward: 0.72
      Mean episode length: 895.62
Episode_Reward/track_lin_vel_xy_exp: 0.5465
Episode_Reward/track_ang_vel_z_exp: 0.3790
      Episode_Reward/lin_vel_z_l2: 0.0000
      Episode_Reward/ang_vel_xy_l2: -0.0673
      Episode_Reward/dof_torques_l2: -0.0013
      Episode_Reward/dof_acc_l2: -0.0538
      Episode_Reward/action_rate_l2: -0.4519
      Episode_Reward/feet_air_time: 0.0049
Episode_Reward/flat_orientation_l2: -0.0256
      Episode_Reward/dof_pos_limits: -0.0238
Episode_Reward/termination_penalty: -0.0417
      Episode_Reward/feet_slide: -0.0445
Episode_Reward/joint_deviation_hip: -0.0352
Episode_Reward/joint_deviation_arms: -0.1046
Episode_Reward/joint_deviation_fingers: -0.0628
Episode_Reward/joint_deviation_torso: -0.0106
      Curriculum/Terrain_levels: 4.7292
Metrics/base_velocity/error_vel_xy: 0.6414
Metrics/base_velocity/error_vel_yaw: 2.4048
      Episode_Termination/time_out: 0.8021
      Episode_Termination/base_contact: 0.1979
-----
      Total timesteps: 24821760
      Iteration time: 2.28s
      Time elapsed: 00:36:36
      ETA: 01:12:10
```

- By ~1000 iterations, mean reward ceases to be negative, signifying it's learning to walk more than fall.
- Episode termination is ~0.8, meaning at this terrain, 80% of robots are walking.
-

Training:

```
#####
Learning iteration 2022/3000

      Computation: 10875 steps/s (collection: 2.075s, learning 0.185s)
      Mean action noise std: 1.21
      Mean value function loss: 0.0165
      Mean surrogate loss: -0.0111
      Mean entropy loss: 49.3575
      Mean reward: 4.52
      Mean episode length: 919.20
Episode_Reward/track_lin_vel_xy_exp: 0.6832
Episode_Reward/track_ang_vel_z_exp: 0.5765
      Episode_Reward/lin_vel_z_l2: 0.0000
      Episode_Reward/ang_vel_xy_l2: -0.0501
      Episode_Reward/dof_torques_l2: -0.0010
      Episode_Reward/dof_acc_l2: -0.0442
      Episode_Reward/action_rate_l2: -0.5686
      Episode_Reward/feet_air_time: 0.0065
Episode_Reward/flat_orientation_l2: -0.0083
      Episode_Reward/dof_pos_limits: -0.0247
Episode_Reward/termination_penalty: -0.0292
      Episode_Reward/feet_slide: -0.0421
Episode_Reward/joint_deviation_hip: -0.0305
Episode_Reward/joint_deviation_arms: -0.1229
Episode_Reward/joint_deviation_fingers: -0.0725
Episode_Reward/joint_deviation_torso: -0.0077
      Curriculum/Terrain_levels: 6.0106
Metrics/base_velocity/error_vel_xy: 0.5313
Metrics/base_velocity/error_vel_yaw: 1.7195
      Episode_Termination/time_out: 0.8208
      Episode_Termination/base_contact: 0.1792
-----
      Total timesteps: 49717248
      Iteration time: 2.26s
      Time elapsed: 01:12:43
      ETA: 00:35:09
```

- By 2000 iterations, mean reward reaches the additive inverse (-4 to 4), signifying rewards for walking surpass negative rewards (falling).
- Terrain levels keep increasing, causing more training required to walk the higher terrains.

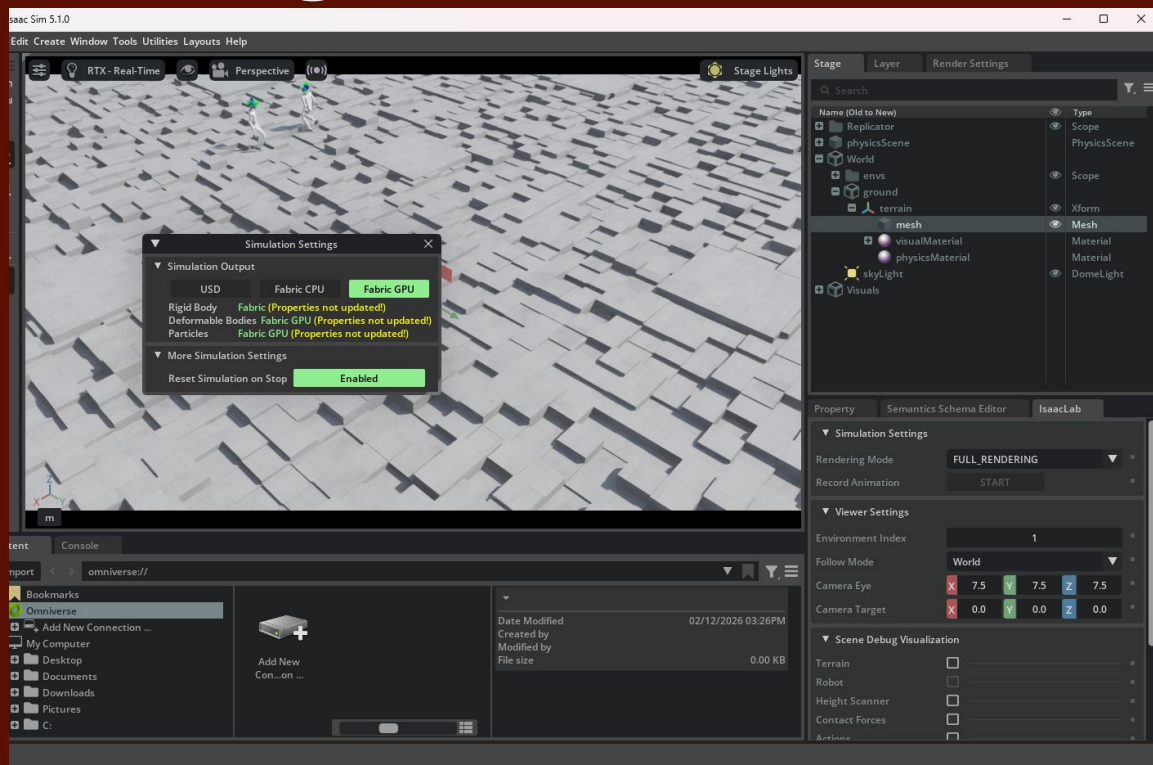
Training:

```
#####
Learning iteration 2990/3000

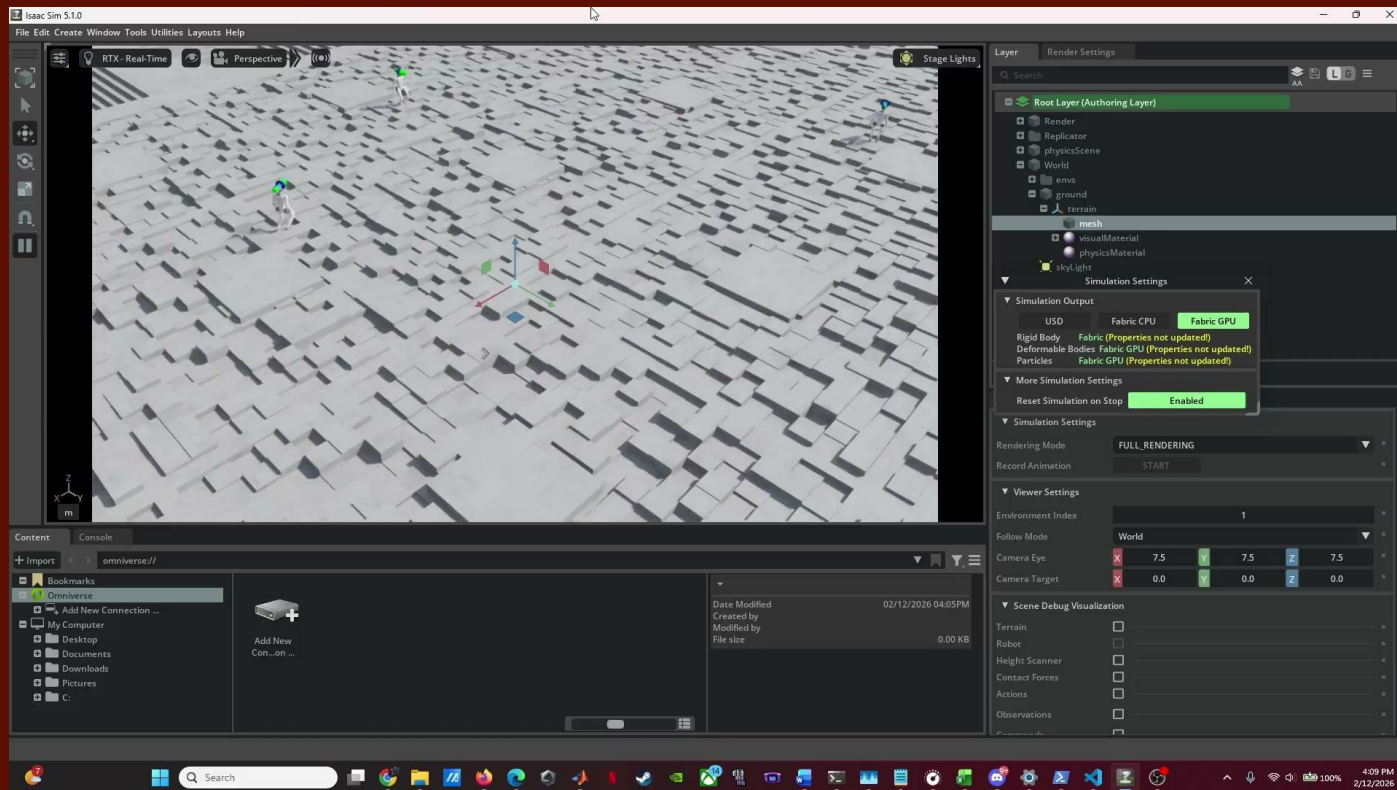
      Computation: 11661 steps/s (collection: 1.920s, learning 0.188s)
      Mean action noise std: 1.24
      Mean value function loss: 0.0202
      Mean surrogate loss: -0.0117
      Mean entropy loss: 48.5442
      Mean reward: 7.57
      Mean episode length: 923.44
Episode_Reward/track_lin_vel_xy_exp: 0.6962
Episode_Reward/track_ang_vel_z_exp: 0.6426
Episode_Reward/lin_vel_z_l2: 0.0000
Episode_Reward/ang_vel_xy_l2: -0.0404
Episode_Reward/dof_torques_l2: -0.0009
Episode_Reward/dof_acc_l2: -0.0380
Episode_Reward/action_rate_l2: -0.5664
Episode_Reward/feet_air_time: 0.0062
Episode_Reward/flat_orientation_l2: -0.0090
Episode_Reward/dof_pos_limits: -0.0261
Episode_Reward/termination_penalty: -0.0333
Episode_Reward/feet_slides: -0.0360
Episode_Reward/joint_deviation_hip: -0.0265
Episode_Reward/joint_deviation_arms: -0.1100
Episode_Reward/joint_deviation_fingers: -0.0722
Episode_Reward/joint_deviation_torso: -0.0089
Curriculum/terrain_levels: 6.0730
Metrics/base_velocity/error_vel_xy: 0.4464
Metrics/base_velocity/error_vel_yaw: 1.3820
Episode_Termination/time_out: 0.8597
Episode_Termination/base_contact: 0.1403
-----
Total timesteps: 73728000
Iteration time: 2.11s
Time elapsed: 01:47:30
ETA: 00:00:02
```

- By the last learning iteration, robot reaches the highest mean reward (7.57), mean episode length (923.44), and episode termination (0.8597) signifying the training was overall a success.

Training:



Training:



Research on IsaacLab

- The primary set of code for training the unitree g1 with rough terrain using Reinforcement Learning consists of:
 - `source/isaacsim_assets/isaacsim_assets/robots/unitree.py`
 - `source/isaacsim_tasks/isaacsim_tasks/manager_based/locomotion/velocity/config/g1/rough_env_cfg.py`
 - `source/isaacsim_tasks/isaacsim_tasks/manager_based/locomotion/velocity/config/g1/agents/rsl_rl_ppo_cfg.py`

unitree.py

- Provides configurations for unitree robots (such as G1, A1)
- This contains spawn configurations, actuators (parameters for effort, stiffness, damping, etc.), and initial state pose.

```
380 G1_MINIMAL_CFG = G1_CFG.copy()
381 G1_MINIMAL_CFG.spawn.usd_path = f"{ISAACLAB_NUCLEUS_DIR}/Robots/Unitree/G1/g1_minimal.usd"
382 """Configuration for the Unitree G1 Humanoid robot with fewer collision meshes.
383
384 This configuration removes most collision meshes to speed up simulation.
385 """
386
387
388 G1_29DOF_CFG = ArticulationCfg(
389     spawn=sim_utils.UsdFileCfg(
390         usd_path=f"{ISAACLAB_NUCLEUS_DIR}/Robots/Unitree/G1/g1.usd",
391         activate_contact_sensors=False,
392         rigid_props=sim_utils.RigidBodyPropertiesCfg(
393             disable_gravity=False,
394             retain_accelerations=False,
395             linear_damping=0.0,
396             angular_damping=0.0,
397             max_linear_velocity=1000.0,
398             max_angular_velocity=1000.0,
399             max_depenetration_velocity=1.0,
400         ),
401         articulation_props=sim_utils.ArticulationRootPropertiesCfg(
402             enabled_self_collisions=False,
403             fix_root_link=False, # Configurable - can be set to True for fixed base
404             solver_position_iteration_count=8,
405             solver_velocity_iteration_count=4,
406         ),
407     ),
408     init_state=ArticulationCfg.InitialStateCfg(
409         pos=(0.0, 0.0, 0.75),
410         rot=(0.7071, 0, 0, 0.7071),
411         joint_pos={
412             ".*_hip_pitch_joint": -0.10,
413             ".*_knee_joint": 0.30,
414             ".*_ankle_pitch_joint": -0.20,
415         },
416         joint_vel={"*": 0.0},
417     ),
```

rough_env_cfg.py

- As discussed last week, this code controls the environment (terrain, what the robot can observe, rewards).

```
C:\Users\Mario> IsaacLab > source > isaacsim_tasks > isaacsim_tasks > manager_based > locomotion > velocity > config > gl > rough_env_cfg.py > ...  
1 # Copyright (c) 2022-2025, The Isaac Lab Project Developers. https://github.com/isaac-sim/IsaacLab/blob/main/CONTRIBUTORS.md.  
2 # All rights reserved.  
3 #  
4 # SPDX-License-Identifier: BSD-3-Clause  
5  
6 from isaacsim.managers import RewardTermCfg as RewTerm  
7 from isaacsim.managers import SceneEntityCfg  
8 from isaacsim.utils import configclass  
9  
10 import isaacsim_tasks.manager_based.locomotion.velocity_mdp as mdp  
11 from isaacsim_tasks.manager_based.locomotion.velocity.velocity_env_cfg import LocomotionVelocityRoughEnvCfg, RewardsCfg  
12  
13  
14 # Pre-defined configs  
15 ##  
16 from isaacsim_assets import GI_MINIMAL_CFG # isort: skip  
17  
18  
19 @configclass  
20 class GLRewards(RewardsCfg):  
21     """Reward terms for the MDP."""  
22  
23     termination_penalty = RewTerm(func=mdp.is_terminated, weight=-200.0)  
24     track_lin_vel_xy_exp = RewTerm(  
25         func=mdp.track_lin_vel_xy_yaw_frame_exp,  
26         weight=1.0,  
27         params={"command_name": "base_velocity", "std": 0.5},  
28     )  
29     track_ang_vel_z_exp = RewTerm(  
30         func=mdp.track_ang_vel_z_world_exp, weight=2.0, params={"command_name": "base_velocity", "std": 0.5}  
31     )  
32     feet_air_time = RewTerm(  
33         func=mdp.feet_air_time_positive_biped,  
34         weight=0.25,  
35         params={  
36             "command_name": "base_velocity",  
37             "sensor_cfg": SceneEntityCfg("contact_forces", body_names=".*_ankle_roll_link"),  
38             "threshold": 0.4,  
39         },  
40     )  
41     feet_slide = RewTerm(  
42         func=mdp.feet_slide,  
43         weight=-0.1,  
44         params={  
45             "sensor_cfg": SceneEntityCfg("contact_forces", body_names=".*_ankle_roll_link"),  
46             "asset_cfg": SceneEntityCfg("robot", body_names=".*_ankle_roll_link"),  
47         },  
48     )  
49  
50     # Penalize ankle joint limits  
51     dof_pos_limits = RewTerm(  
52         func=mdp.joint_pos_limits,  
53         weight=-1.0,  
54         params={"asset_cfg": SceneEntityCfg("robot", joint_names=[".*_ankle_pitch_joint", ".*_ankle_roll_joint"])},  
55     )
```

G1RoughPPORunnerCfg

- The Reinforcement Learning Process. Contains useful parameters such as:
 - Hidden Layers
 - Learning Rate
 - Schedule Type (how it increases difficulty – in this case the terrain).

```
C:\Users\Mario> isaacsim > source > isaacsim_tasks > isaacsim_tasks > manager_based > locomotion > velocity > config > g1 > agents >  rsl_rl_ppo_cfg.py >  G1RoughPPORunnerCfg

1 # Copyright (c) 2022-2025, The Isaac Lab Project Developers.
2 # SPDX-License-Identifier: BSD-3-Clause
3
4 from isaacsim.utils import configclass
5 from isaacsim.rl.rsl_rl import RslRlOnPolicyRunnerCfg, RslRlPpoActorCriticCfg, RslRlPpoAlgorithmCfg
6
7 @configclass
8 class G1RoughPPORunnerCfg(RslRlOnPolicyRunnerCfg):
9     num_steps_per_env = 24
10     max_iterations = 3000
11     save_interval = 50
12     experiment_name = "g1_rough"
13
14     # This is the "Dual-Link" config to satisfy both Isaac Lab and RSL-RL
15     policy = RslRlPpoActorCriticCfg(
16         class_name="ActorCritic",
17         init_noise_std=1.0,
18         actor_obs_normalization=False,
19         critic_obs_normalization=False,
20         actor_hidden_dims=[512, 256, 128],
21         critic_hidden_dims=[512, 256, 128],
22         activation="elu",
23     )
24
25     algorithm = RslRlPpoAlgorithmCfg(
26         value_loss_coef=1.0,
27         use_clipped_value_loss=True,
28         clip_param=0.2,
29         entropy_coef=0.008,
30         num_learning_epochs=5,
31         num_mini_batches=4,
32         learning_rate=1.0e-3,
33         schedule="adaptive",
34         gamma=0.99,
35         lam=0.95,
36         desired_kl=0.01,
37         max_grad_norm=1.0,
38     )
39
40 @configclass
41 class G1FlatPPORunnerCfg(G1RoughPPORunnerCfg):
42     def __post_init__(self):
43         super().__post_init__()
44
45         self.max_iterations = 1500
46         self.experiment_name = "g1_flat"
47         self.policy.actor_hidden_dims = [256, 128, 128]
48         self.policy.critic_hidden_dims = [256, 128, 128]
```

Next Steps

- To learn more about how to utilize the Isaac Labs + Isaac Sim folder.
 - How do do the exact same project with the r1 robot?
- Understand more about Isaac Lab's reinforcement learning environment
 - How create RL environments to train g1 (r1 in the future) robot to do different tasks.
- Might try to complete Isaac Gym's RL environment to compare with Isaac Labs

Sources

- https://github.com/isaac-sim/IsaacLab/blob/main/source/isaacab_tasks/isaacab_tasks/manager_based/locomotion/velocity/config/g1/agents/rsl_rl_ppo_cfg.py
-